武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第一讲 分布式软件概述

计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 什么是分布式软件?

什么是分布式软件?

分布式软件

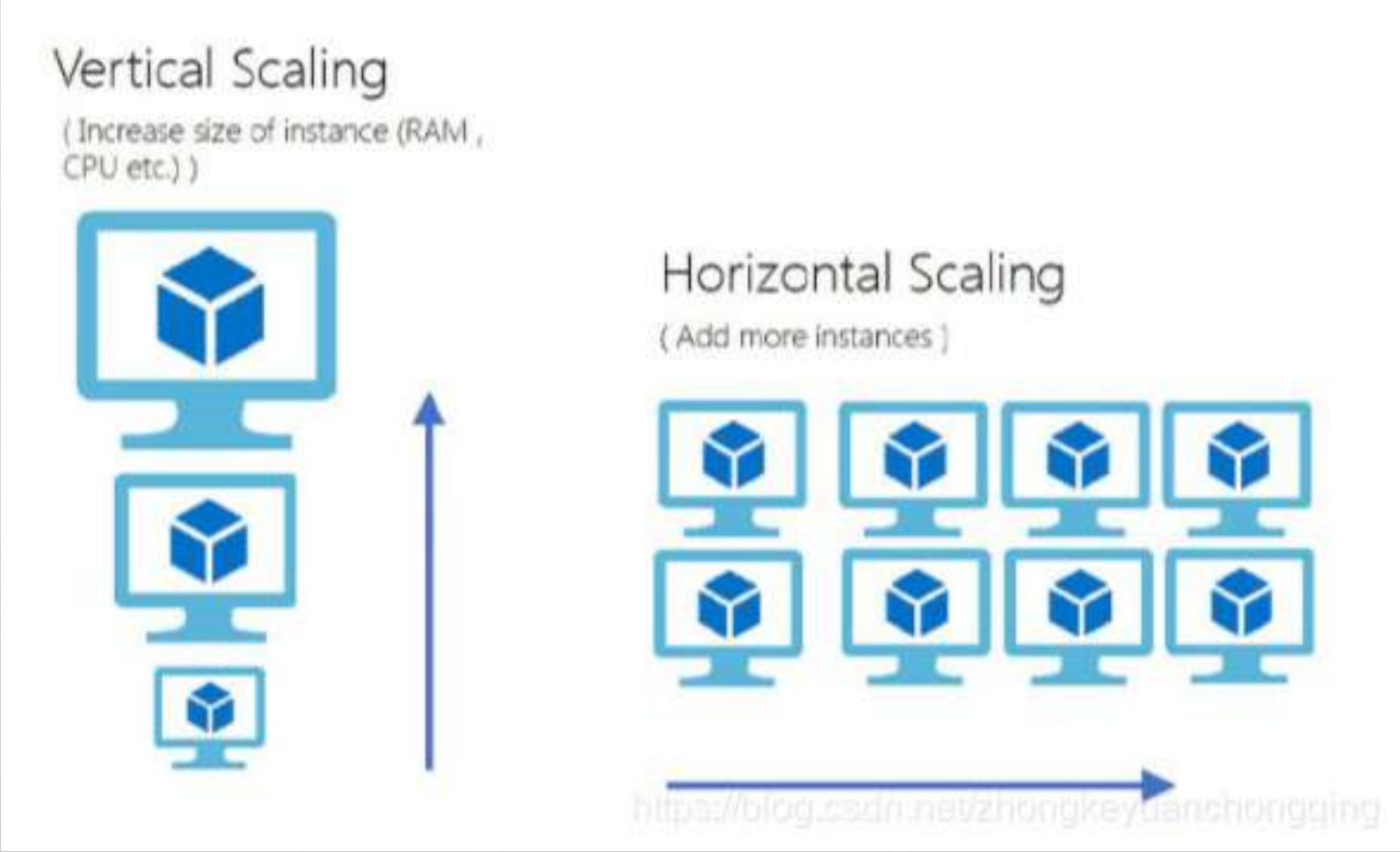
- 分布式软件是指其组件分布在通过网络连接的不同计算机（节点）上，它们通过传递消息进行通信和协调，从而共同完成一个任务的软件系统

核心本质

- **物理上的分散**：硬件或软件组件位于通过网络连接的不同计算机（或虚拟机/容器）中
- **消息传递通信**：节点之间没有共享内存，通过“传纸条”（发送网络消息）交流信息
- **共同目标**：所有节点为了完成同一个任务（比如支撑双11的流量）而存在

为什么要搞分布式?

当任务规模扩大或计算复杂的增高，单机的垂直扩展无法支撑，需要多台机器实现水平扩展



阿里“去IOE”历程

- 第一阶段：战略启动（2008-2009）
  - 2008年：阿里巴巴正式提出“去IOE”战略构想
  - 2009年：阿里云成立；王坚博士决策正式启动“去IOE”工程
  - 战略动因：业务爆发式增长倒逼技术突破，传统集中式IOE架构无法支撑
- 第二阶段：攻坚落地（2010-2012）
  - 2010年5月：OceanBase立项，开启数据库自研之路
  - 2010年7月：淘宝商品库完成“去I”（去IBM小型机）
  - 2011年7月：淘宝商品库完成“去OE”（去Oracle+EMC）
  - 2011年9月：淘宝交易库完成“去IOE”
- 第三阶段：标志性胜利（2013）
  - 2013年5月17日：支付宝最后一台IBM小型机下线
  - 2013年7月10日：淘宝最后一个Oracle数据库下线（广告系统）
  - 2013年8月：飞天5K集群发布（单集群5000台服务器）
  - 2013年双十一：实现零漏单、零故障，验证分布式架构可靠性

分布式软件的目标

- 分布式的核心目标： “三高”
  - **高并发：**原来1个机器支持500个并发请求，现在用100台机器支持五万个并发
  - **高性能：**把 1 个需要跑 10 天的任务，拆给 10 个机器跑，1 天就能跑完。
  - **高可用：**一家银行 10 台服务器，坏了 1 台，另外 9 台还能继续工作。

分布式的副作用

- 分布式不是免费的午餐，它引入了极高的复杂度
  - **可靠性**：网络被“切断”（网络分区）,消息可能会丢失、延迟、乱序
  - **一致性**：你在 A 节点改了余额，B 节点能不能立刻看到？如果不能，用户就会看到错误的余额

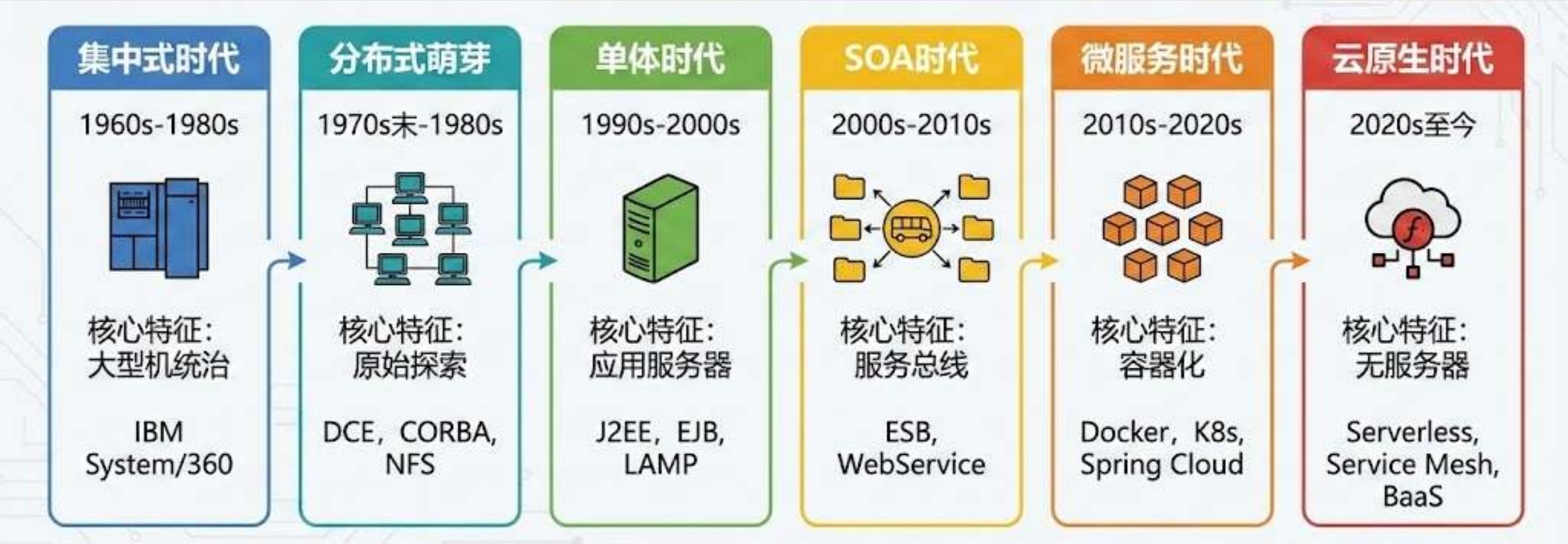
- 分布式是为了解决单机无法承载的规模化问题，和单机无法提供的容错性问题
- 不能为了分布式而分布式，否则系统将变得复杂而脆弱

分布式相关术语内涵

术语	范畴	定义	解决的关键问题
分布式计算	计算范式	将一个巨大的计算任务拆分成多个小的子任务，分配给网络中的多台计算机同时处理，最后汇总结果。如MapReduce、Spark等分布式计算框架，DeepSpeed、Ray等分布式训推框架	任务切分、并行处理等计算优化问题
分布式系统	软硬件集合体	硬件或软件组件分布在不同的网络计算机上，彼此之间仅仅通过消息传递进行通信和协调	节点、网络、局部失效、逻辑时钟等问题
分布式架构	系统设计	分布式系统内部各组件如何组织、如何交互的结构化设计方案。如主从架构、对等架构、微服务架构、无服务器架构 (Serverless)等	拓扑结构、通信协议解耦、服务治理等问题
分布式软件	软件产品	运行在分布式系统之上的应用软件（如Google 搜索、微信、淘宝），由多个协同工作的组件（如前端、后端服务、数据库集群）组成的软件集合	高并发、高性能、高可用、高可靠（一致性）

2、 分布式软件发展史

分布式架构的历史


集中式时代
1960s-1980s
核心特征：
大型机统治
IBM
System/360
分布式萌芽
1970s末-1980s
核心特征：
原始探索
DCE，CORBA，
NFS
单体时代
1990s-2000s
核心特征：
应用服务器
J2EE，EJB，
LAMP
SOA时代
2000s-2010s
核心特征：
服务总线
ESB，
WebService
微服务时代
2010s-2020s
核心特征：
容器化
Docker，K8s，
Spring Cloud
云原生时代
2020s至今
核心特征：
无服务器
Serverless，
Service Mesh，
BaaS

分布式萌芽（1970s-1980s）

- 时代背景
  - 微型计算机兴起（Intel 8086，1978年）
  - 单机算力不足，催生多机协作需求
- 历史教训
  - 追求“透明分布式调用”的理想过于超前，网络延迟、分区、容错等问题使复杂度远超收益

技术（Tech）	提出者（Origin）	现代影响（Legacy）
NCA	惠普/Apollo	RPC雏形
AFS	CMU	分布式文件系统鼻祖
Kerberos	MIT	服务认证基础协议

里程碑：DCE（分布式运算环境）

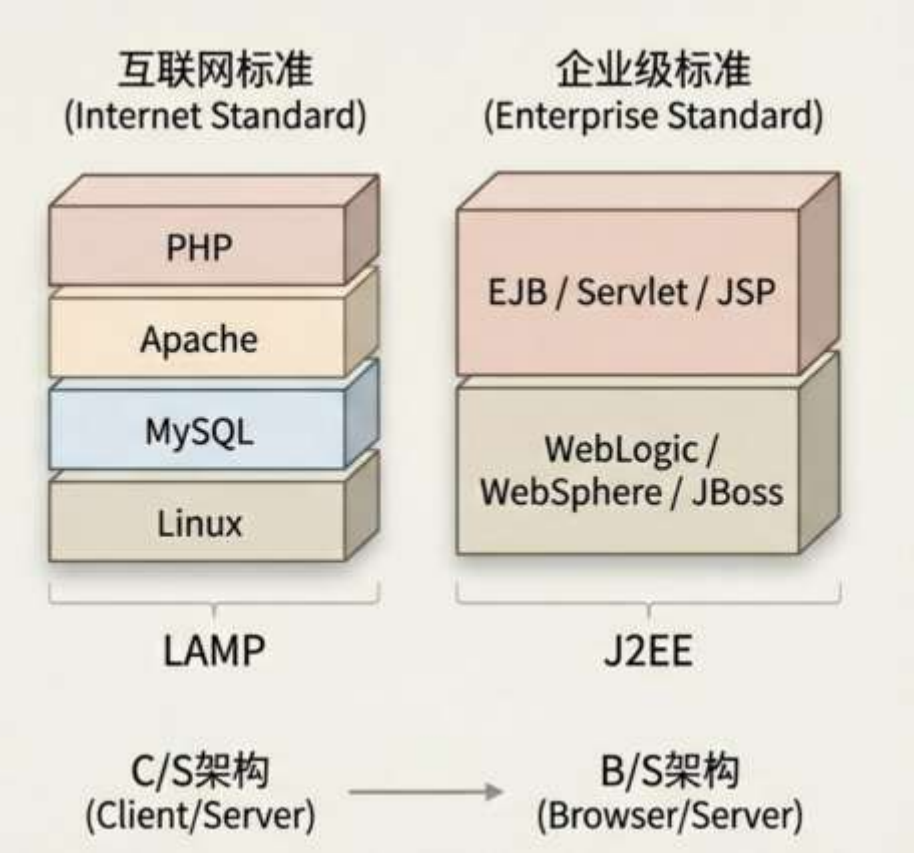
由开放软件基金会制定，包含RPC、DFS、UUID等规范。

分布式组件时代（1990s-2000s）

- 1992年，OMG 组织推出了CORBA
- 试图通过“分布式对象”，实现跨语言、跨平台的宏大统一。
- 由于违背了简单的原则，高昂的学习成本和厂商间的不兼容导致其最终败给了更简单的Web 技术。

单体时代(1990s-2010s)

- 背景：互联网应用的崛起、摩尔定律的硬件红利。
- 技术：简单高效的单体架构



“单体”不是“单机”，也可以是分布式软件！

架构演进路径：

1.单应用单数据库：所有功能在一台服务器

2.应用与数据库分离：独立部署

3.应用服务器集群：引入负载均衡

4.数据库读写分离：主从复制

5.引入缓存：Redis/Memcached

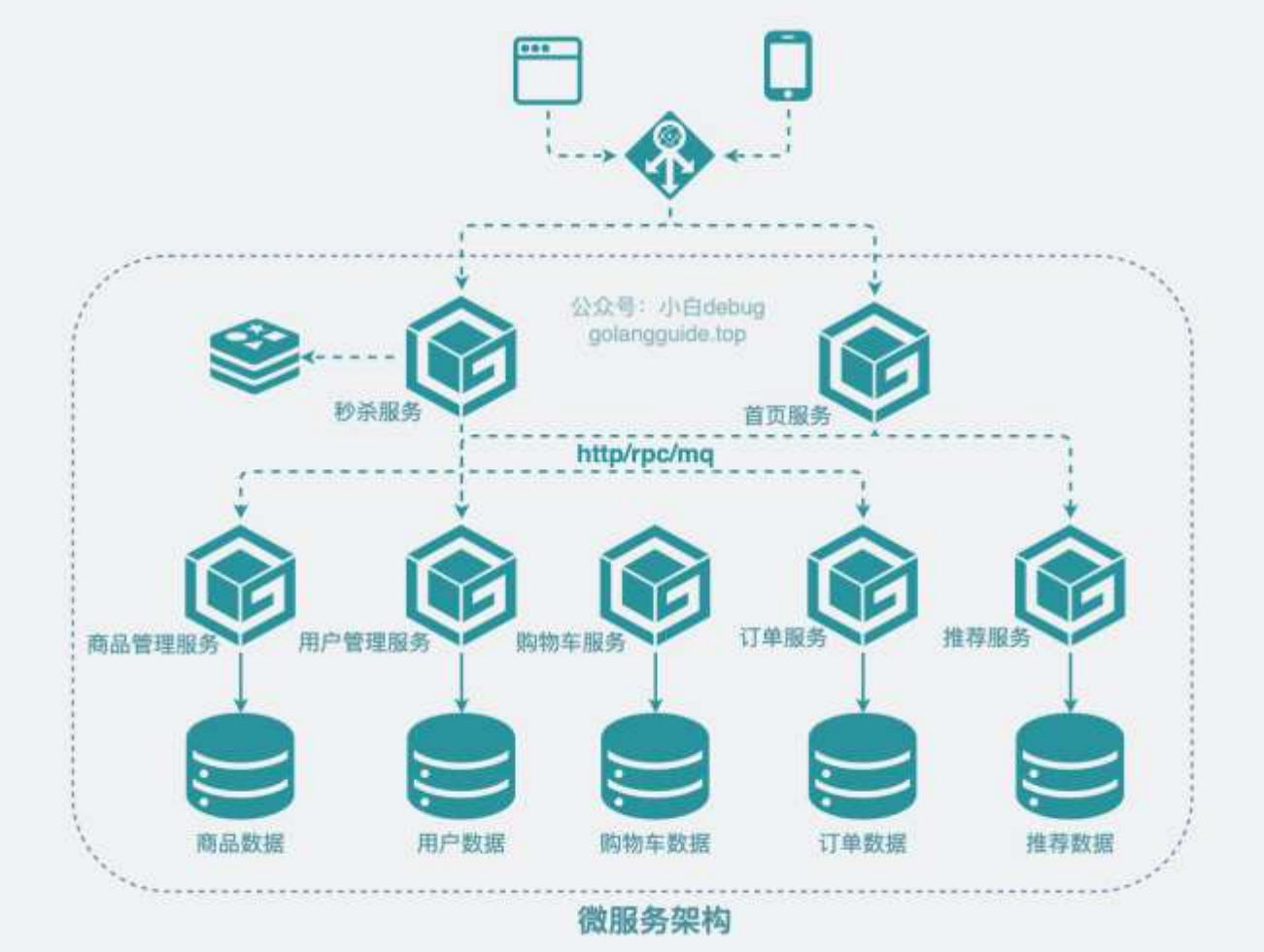
6. 数据库垂直/水平拆分：分库分表

SOA时代（2000s-2010s）

- **背景：**异构应用集成
- **技术：**企业试图通过ESB（企业服务总线）避免点对点集成的复杂，使用SOAP协议交互，来解决异构系统集成。
- **缺点：**ESB 成为了新的性能瓶颈和单点故障；SOAP协议笨重低效，向轻量级 RESTful 协议转型。

微服务时代（2010s-2020s）

- **背景：**应用规模逐步增大，单体应用的开发运维效率低下
- **理念**：将单体拆分为独立部署的小服务，每个服务拥有独立的数据库，通过轻量级API通信。
- **优点**：每个服务由独立的开发团队进行开发与测试。解决了分工和开发效率问题。
- **缺点**：增加了开发（如分布式事务）和运维（链路追踪）复杂性。



微服务时代（2010s-2020s）

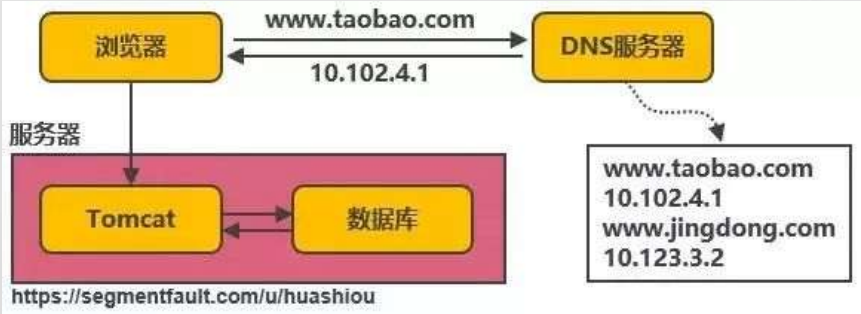
- **服务治理**：服务数量的激增，对服务治理带来了巨大挑战
- 容器化成为微服务的标准化部署和治理方式。
- 容器编排平台（如Kubernetes）支持基于容器实现各种自动化的治理

	Kubernetes	Spring Cloud
弹性伸缩	Autoscaling	N/A
服务发现	KubeDNS / CoreDNS	Spring Cloud Eureka
配置中心	ConfigMap / Secret	Spring Cloud Config
服务网关	Ingress Controller	Spring Cloud Zuul
负载均衡	Load Balancer	Spring Cloud Ribbon
服务安全	RBAC API	Spring Cloud Security
跟踪监控	Metrics API / Dashboard	Spring Cloud Turbine
降级熔断	N/A	Spring Cloud Hystrix

云原生时代（2020s以后）

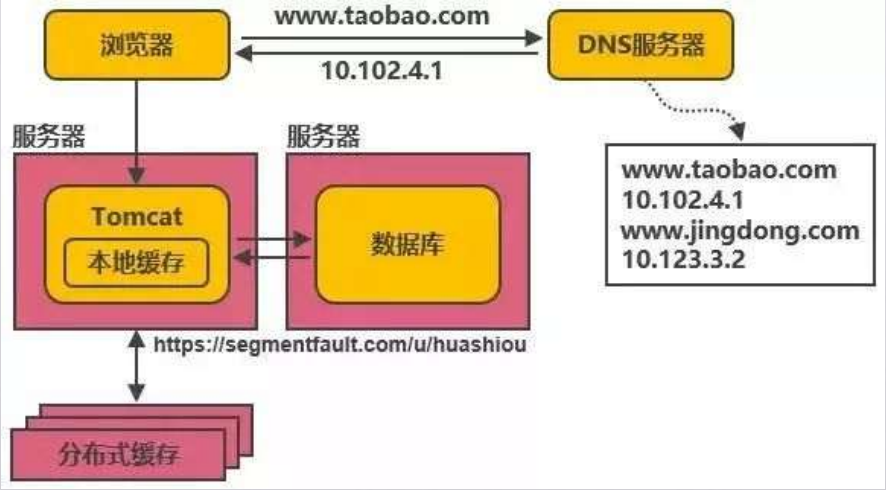
- Service Mesh(服务网格)：服务治理下层到基础设施（Sidecar模式）中，对开发者透明
- Backend as a Service(后端即服务)：数据库、消息队列、日志、存储等等后端服务直接在云上提供
- Serverless(无服务器架构)：提供了函数即服务（Function as a Service）能力，支持scale to zero，节省大量运营成本

淘宝的架构演进之路

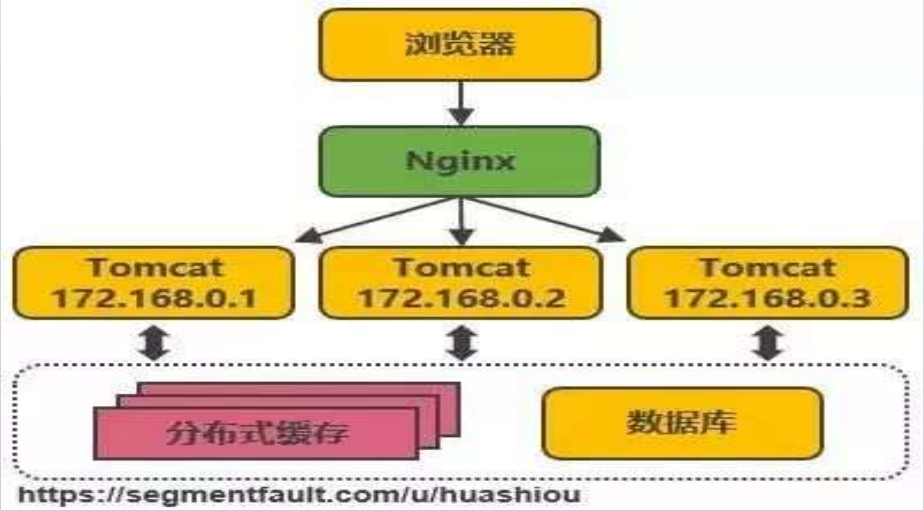


（1）单机架构

（2）Tomcat与数据库分开部署



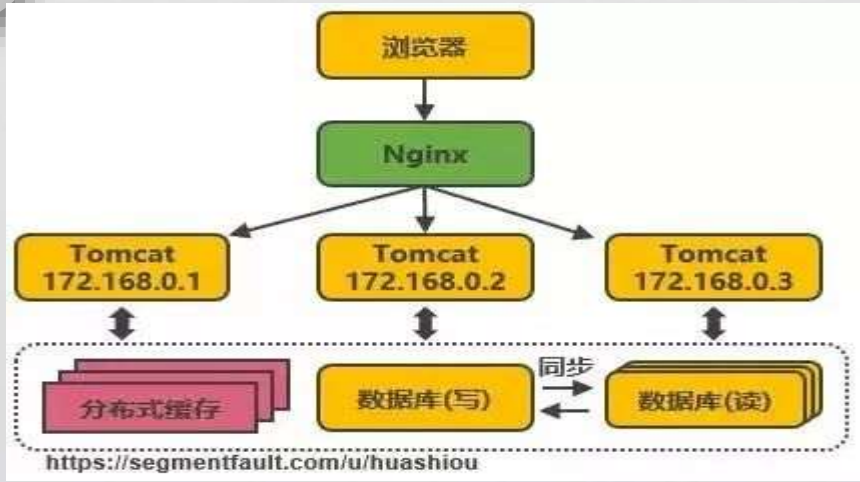
（3）引入缓存



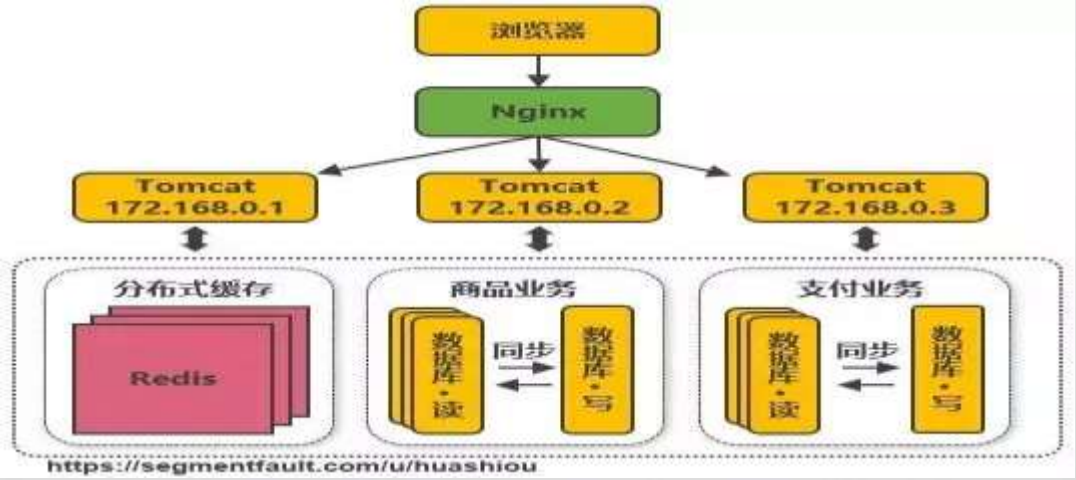
（4）引入反向代理与负载均衡

武漢大學
WUHAN UNIVERSITY

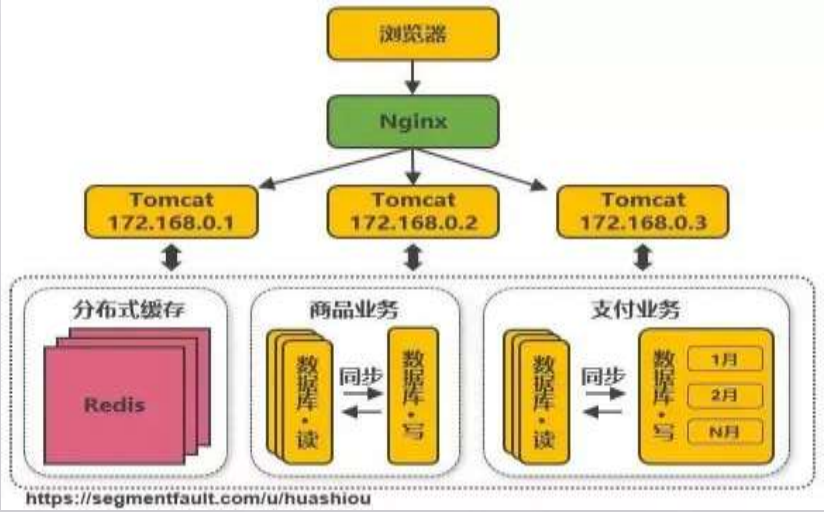
淘宝的架构演进之路



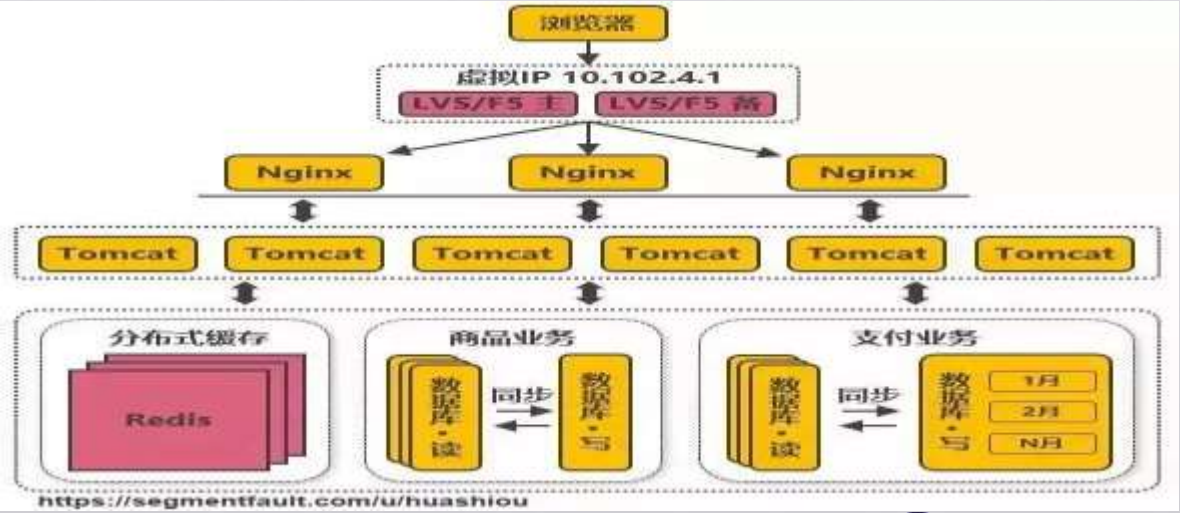
（5）数据库读写分离



（6）按业务分库



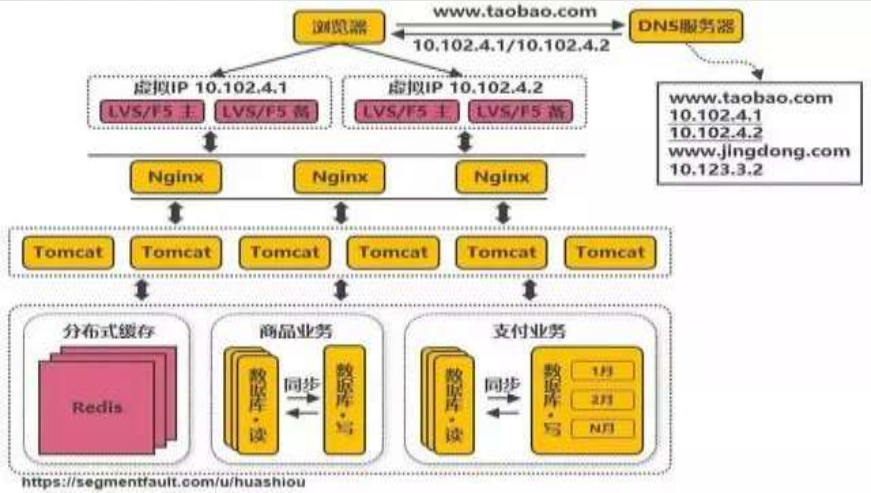
（7）数据库分表



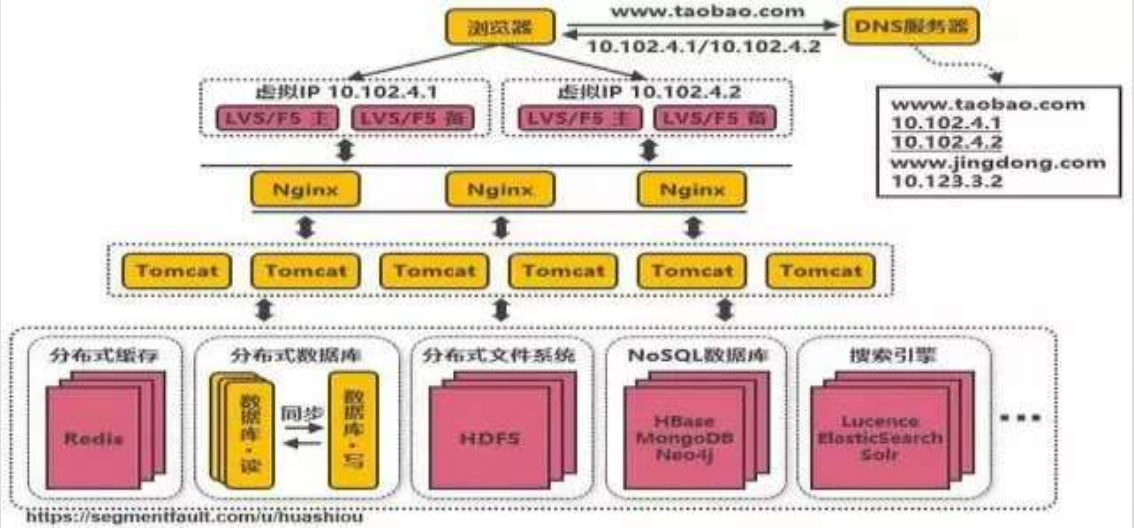
（4）LVS/FS多路负载均衡

武漢大學 WUHAN UNIVERSITY

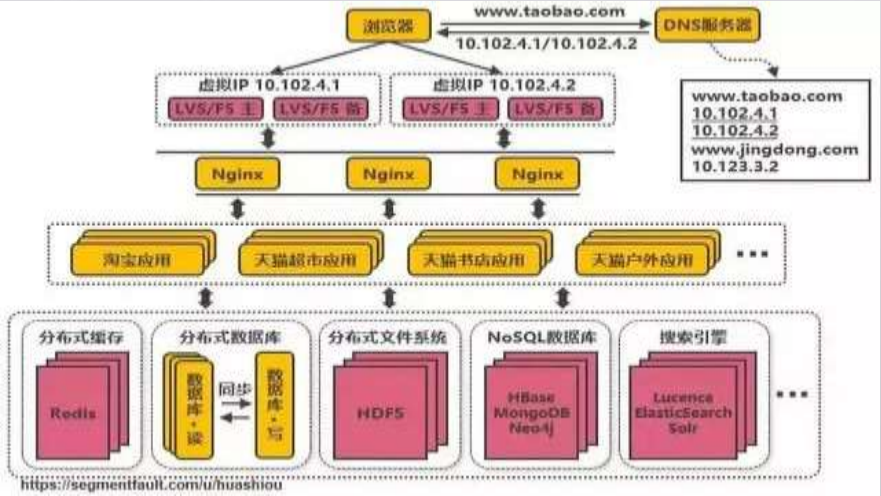
淘宝的架构演进之路



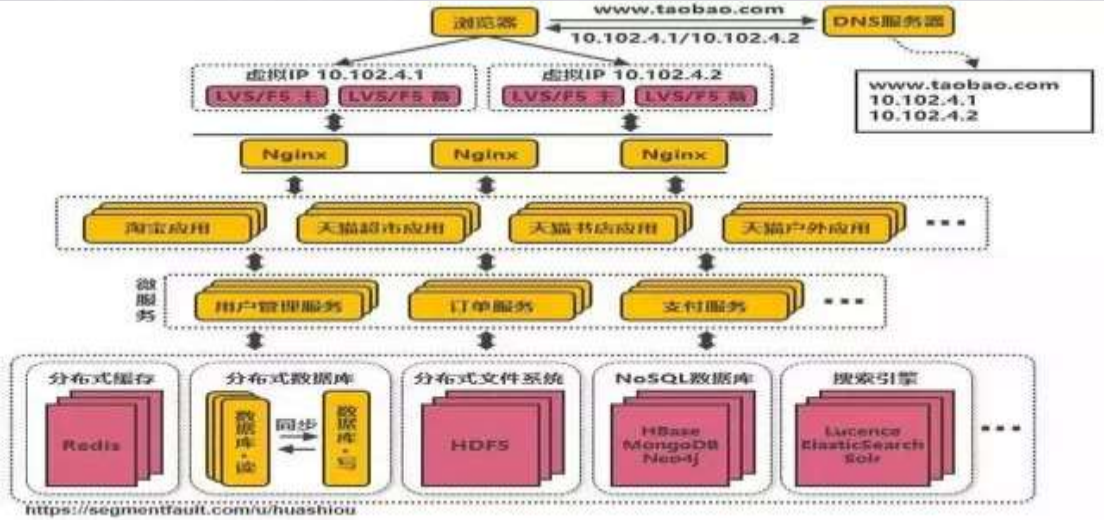
（9）DNS层负载均衡



（10）引入NoSQL数据库和搜索引擎



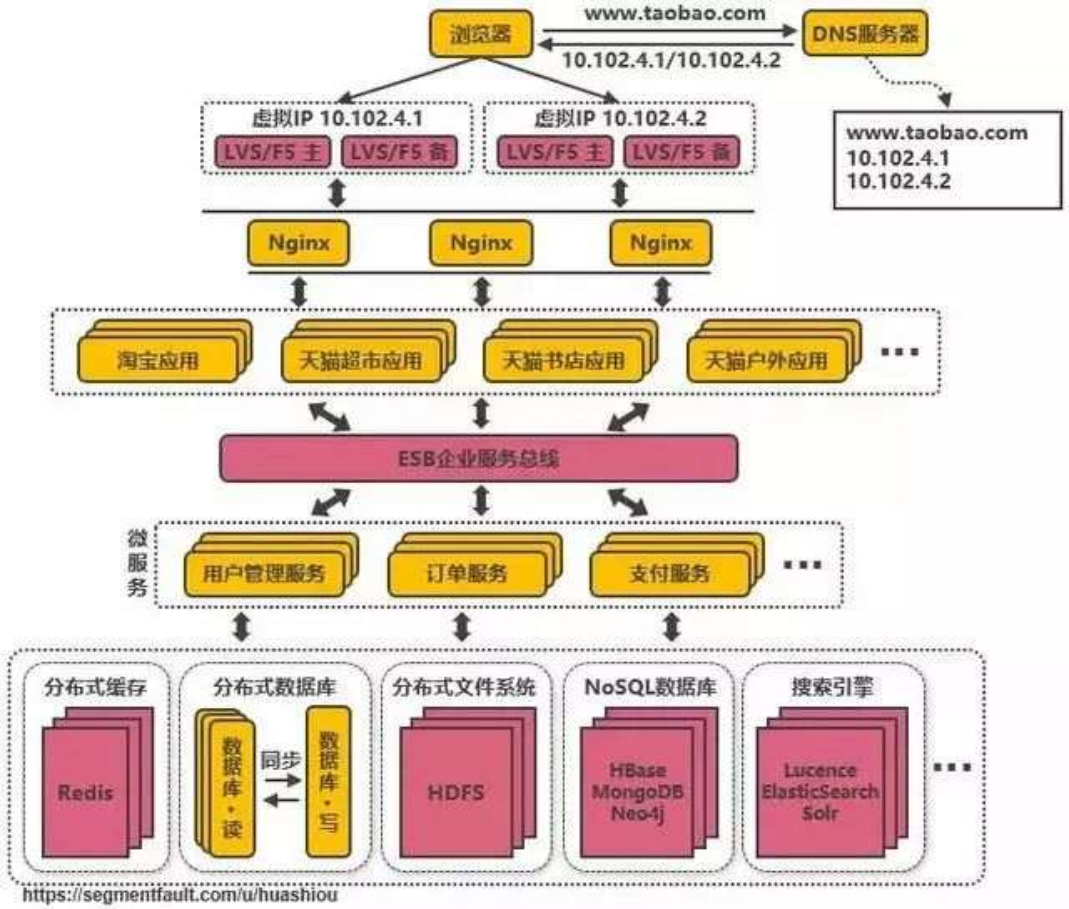
（11）应用分解



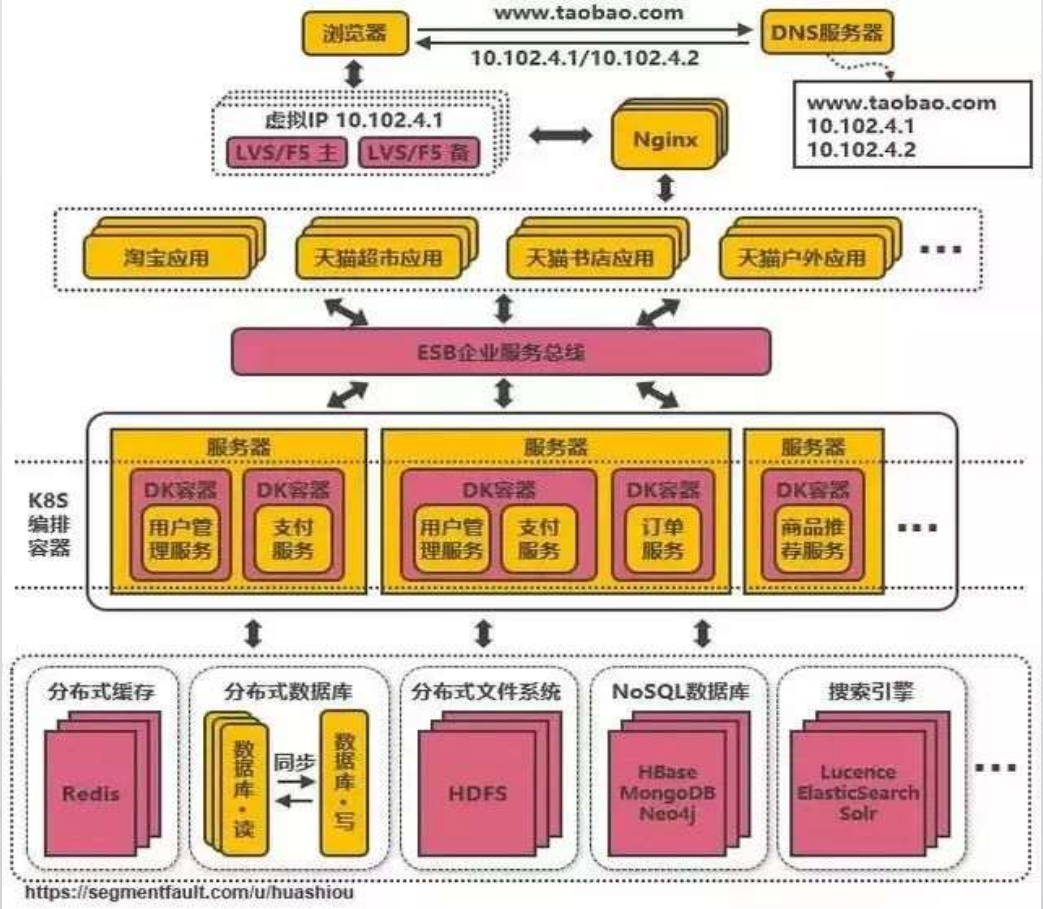
（12）引入微服务架构

WUHAN UNIVERSITY

淘宝的架构演进之路



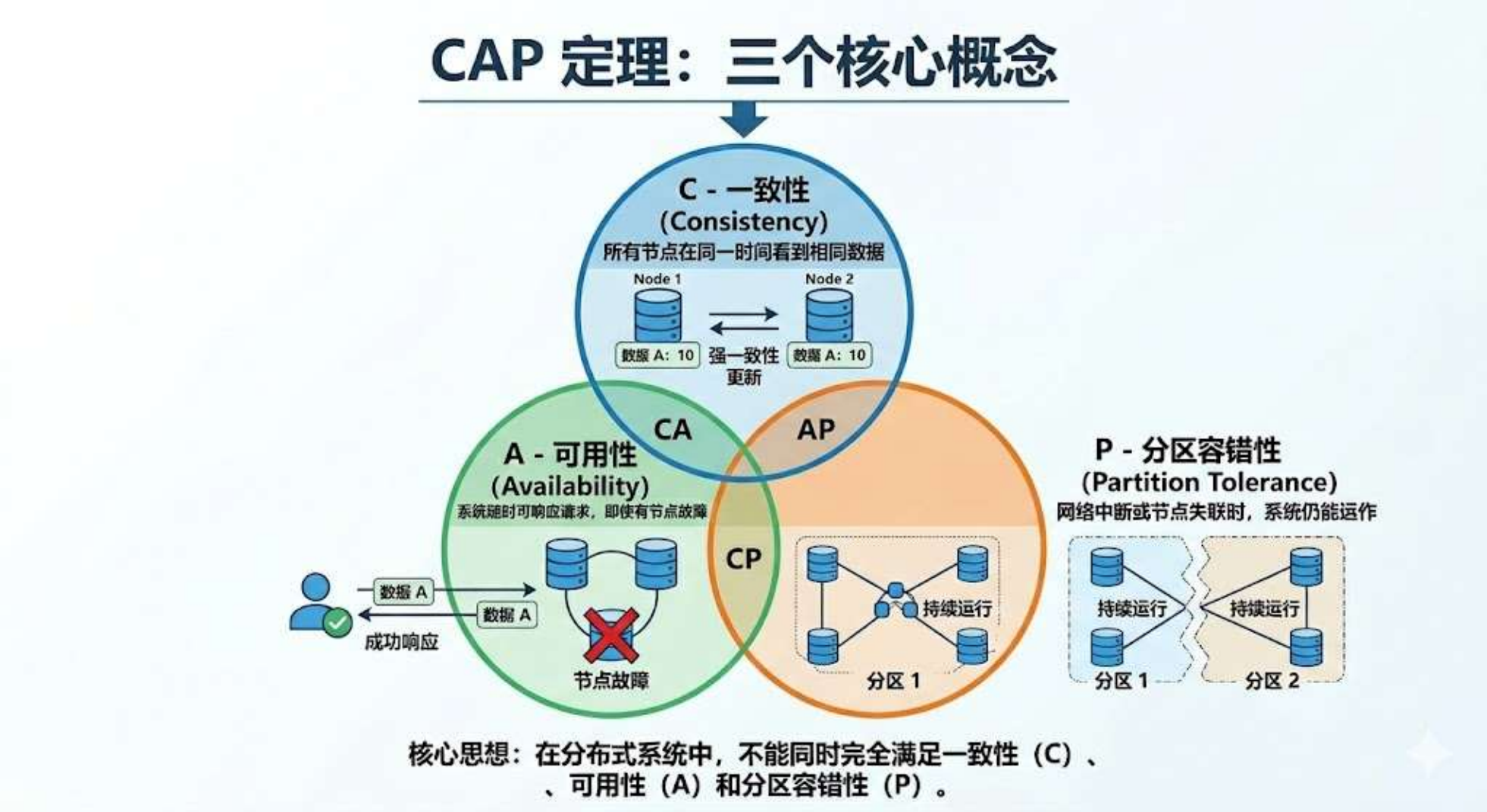
（13）引入服务总线



（14）引入容器与K8S

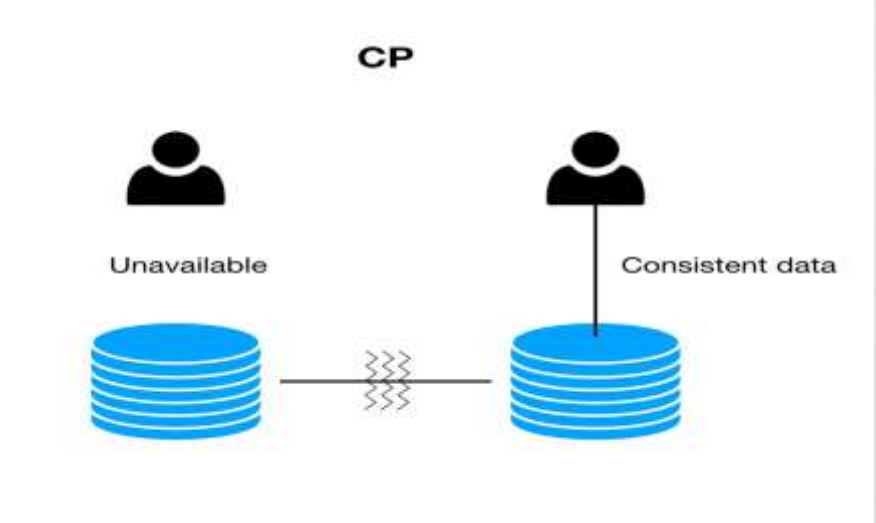
3、 分布式理论

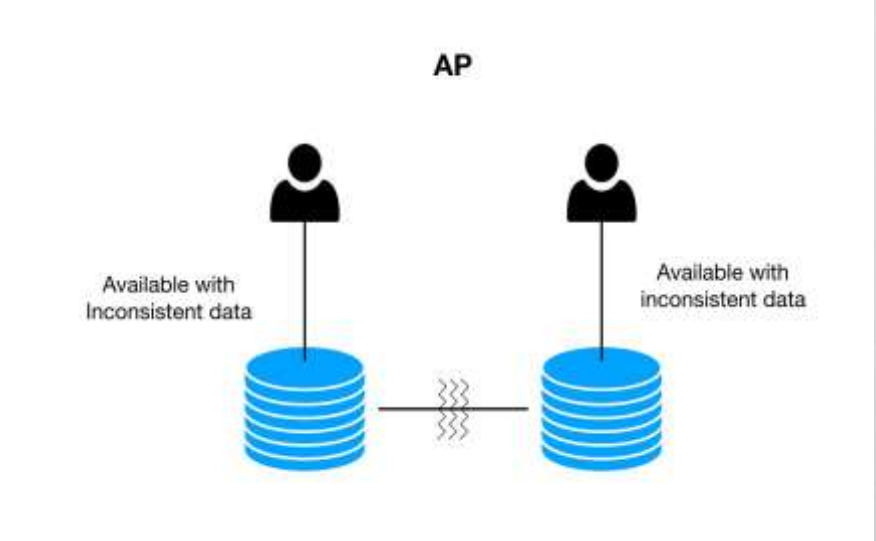
CAP 定理


CAP 定理：三个核心概念
C - 一致性
(Consistency)
所有节点在同一时间看到相同数据
Node 1
Node 2
数据 A：10
强一致性
更新
数据 A：10
CA
AP
A - 可用性
(Availability)
系统随时可响应请求，即使有节点故障
数据 A
数据 A
成功响应
节点故障
CP
持续运行
分区 1
P - 分区容错性
(Partition Tolerance)
网络中断或节点失联时，系统仍能运作
持续运行
持续运行
分区 1
分区 2
核心思想：在分布式系统中，不能同时完全满足一致性（C）、
、可用性（A）和分区容错性（P）。

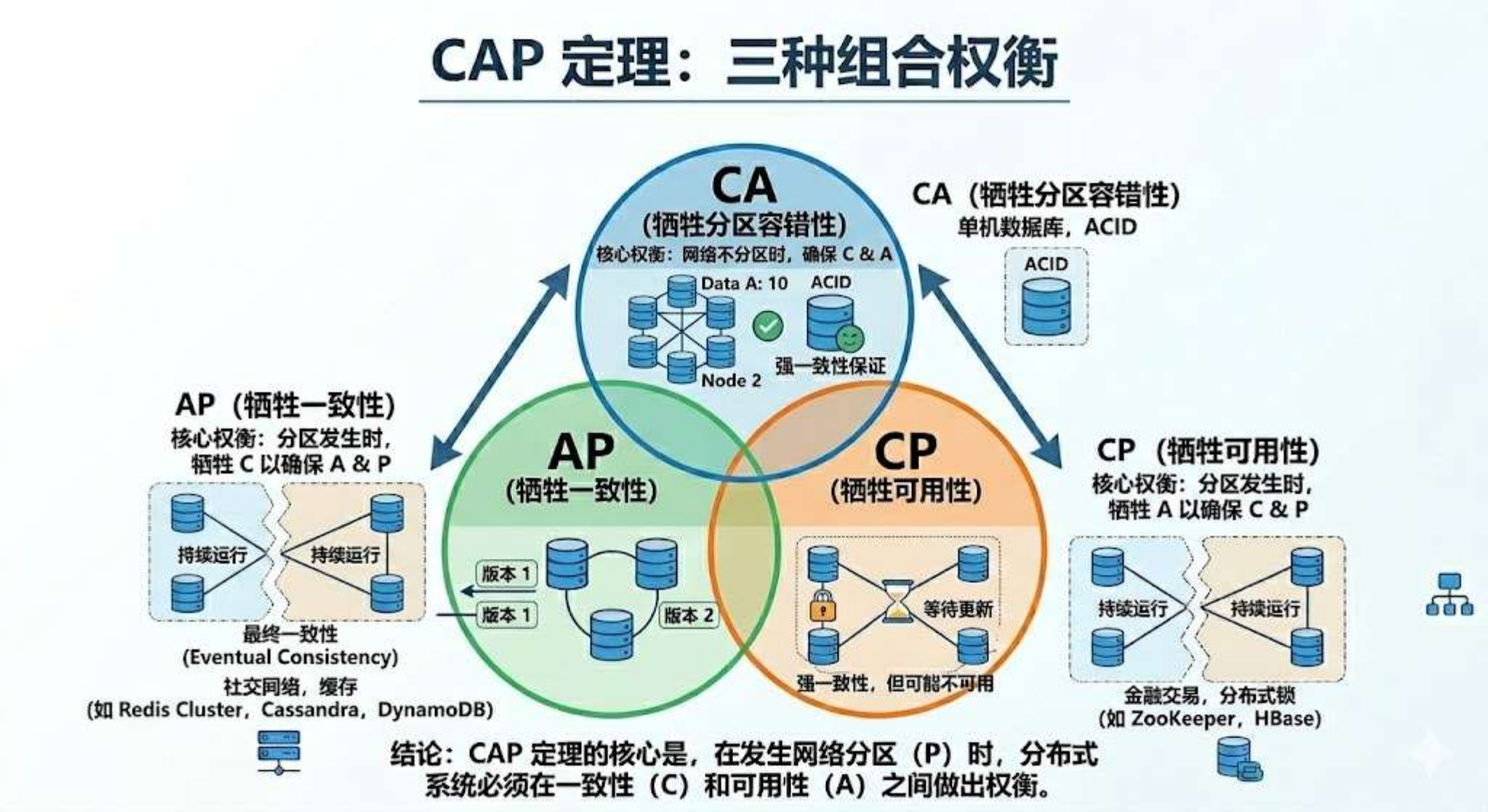
CAP 定理

- **如果考虑P，则需要在AC之间二选一**
  - **CP→!A**：如果为了保证数据一致性（C），数据只存放在一个节点或两节点实时同步，网络断开（P）会造成另一节点不可用，即丧失了A性质。
  - **AP→!C**：如果数据存放在两个节点，网络断开（P）两个节点可以分别读写数据（A），会导致两个节点数据不一致，即丧失了C性质。
- **如果不考虑P，才可以CA兼得**
  - **AC→!P**：如果既保证C又保证A，要求两个节点一直可以互相通信，这又会导致丧失P性质。





CAP 定理



WUHAN UNIVERSITY

CAP 定理

- 如果网络分区客观存在，必须保障P，架构师只能在 CP和 AP之间做权衡
- CP（弃 A）类型
  - 方法：为了保证数据一致，在网络恢复前，某些节点必须拒绝请求。
  - 场景：如银行系统、企业应用
- AP（弃 C）类型
  - 方法为了保证可用，每个节点继续处理请求，但不同分区之间的数据会出现不一致
  - 场景：如社交媒体点赞、购物车、新闻推送

PACELC定理

- PACELC 是对 CAP 的补充，涵盖了在“故障”与“正常”两种状态下的抉择
- 公式拆解： P-A-C / E-L-C
  - P-A-C：如果发生分区，系统必须在 A (Availability) 和 C (Consistency) 之间权衡。
  - E-L-C: 否则（正常情况下），系统必须在 L (Latency) 和 C (Consistency) 之间权衡。

策略类型	典型代表	核心特点
PA/EL	Dynamo, Cassandra	出现分区选可用；正常时选低延迟（牺牲强一致性）。
PC/EC	BigTable, HBase	始终追求强一致性。分区时宁可不可用；正常时宁可慢点。
PC/EL	MongoDB (默认)	分区时选一致性；但正常情况下为了性能选低延迟。
PA/EC	VoltDB	分区时保可用；但正常情况下维持强一致（开销较大）。

PACELC定理

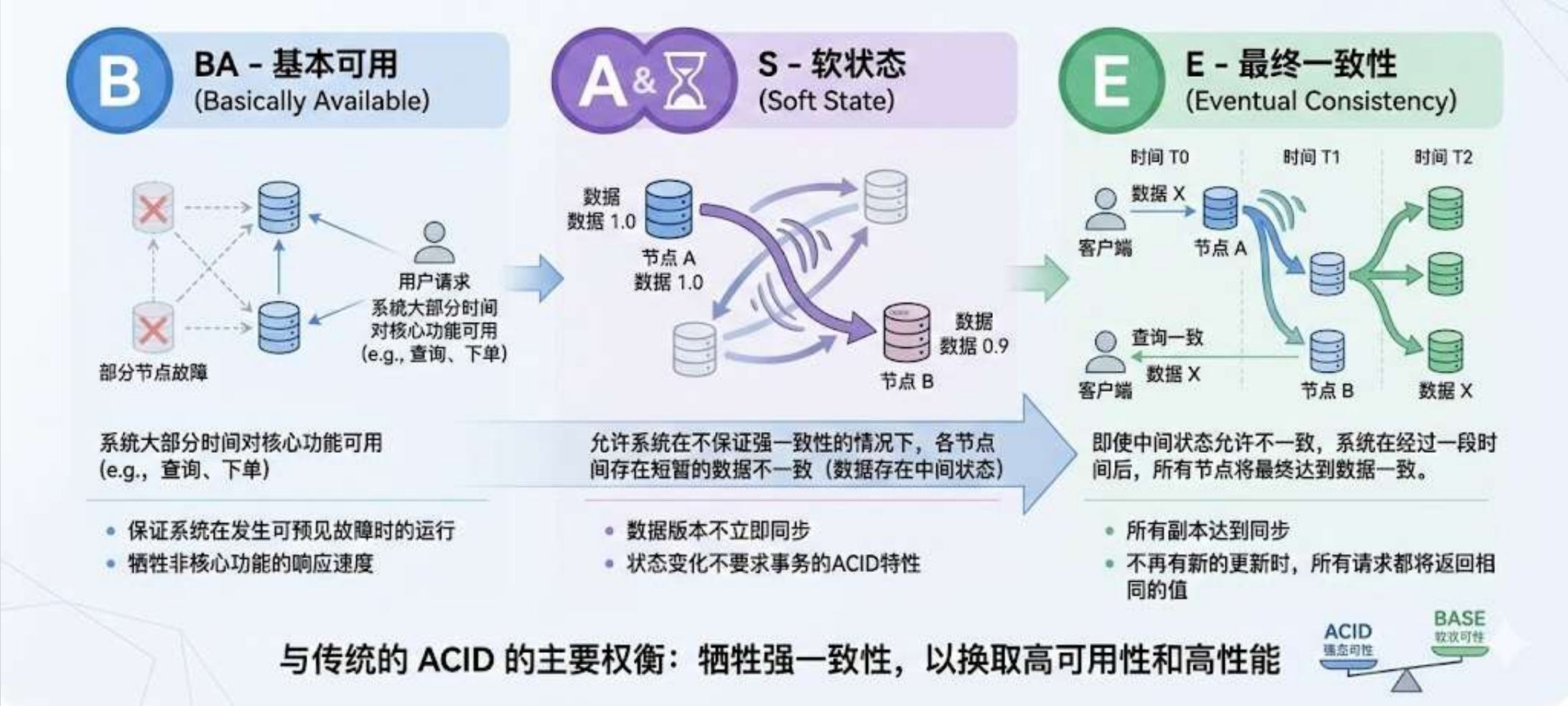
➢ PACELC定理的要点

- CAP处理故障，PACELC覆盖日常
  - 故障时的选择 + 正常运行时的选择 = 完整设计空间。
- 延迟是分布式系统的第一性能指标
  - 强一致往往意味着高延迟，需要在业务容忍度内做取舍。
- 可调一致性是现代分布式平台的标配
  - 让用户根据不同场景选择不同的一致性级别，实现灵活权衡。

大厂面试常见问题

- **常见问题1**
  - Q：“CAP和PACELC什么关系？”
  - A：“PACELC是CAP的扩展，增加了正常运行时延迟与一致性的权衡。”
- **常见问题2**
  - Q：“你设计的系统属于PACELC的哪一类？为什么？
  - A：“我设计的XX系统属于PA/EL，因为业务允许最终一致性，但要求低延迟用户体验，分区时我们选择高可用。”
- **常见问题3**
  - Q：“为什么有些系统在正常运行时也选择弱一致性？”
  - A：“为了降低延迟。强一致通常需要同步复制，增加响应时间。在互联网场景，延迟直接影响用户体验和转化率。”

AP系统的实践指南：BASE理论



AP系统的实践指南：BASE理论

1. BASE是AP系统的设计哲学

- 当我们在CAP中选择了可用性(A)，就用BASE来指导设计
- 接受短暂不一致，换取高可用和高性能

2. 最终一致性不是牺牲一致性

- 它是有保障的一致性，只是需要时间
- 系统设计要保证这个时间在业务可接受范围内

3. 没有银弹，只有权衡

- 强一致性系统：简单正确但慢
- 最终一致性系统：复杂但快

BASE系统与ACID系统的比较

维度	ACID系统（CP系统的典型实现）	BASE系统（AP系统的典型实现）
一致性	强一致性	最终一致性
可用性	低（维护一致性时可能不可用）	高（基本可用）
事务	复杂事务支持	简单事务或补偿事务
响应时间	较慢	较快
设计复杂度	相对简单	相对复杂（要处理不一致）
典型系统	MySQL, PostgreSQL	Cassandra, Dynamo, Redis
适用场景	转账、支付、库存扣减	社交、推荐、购物车、会话数据

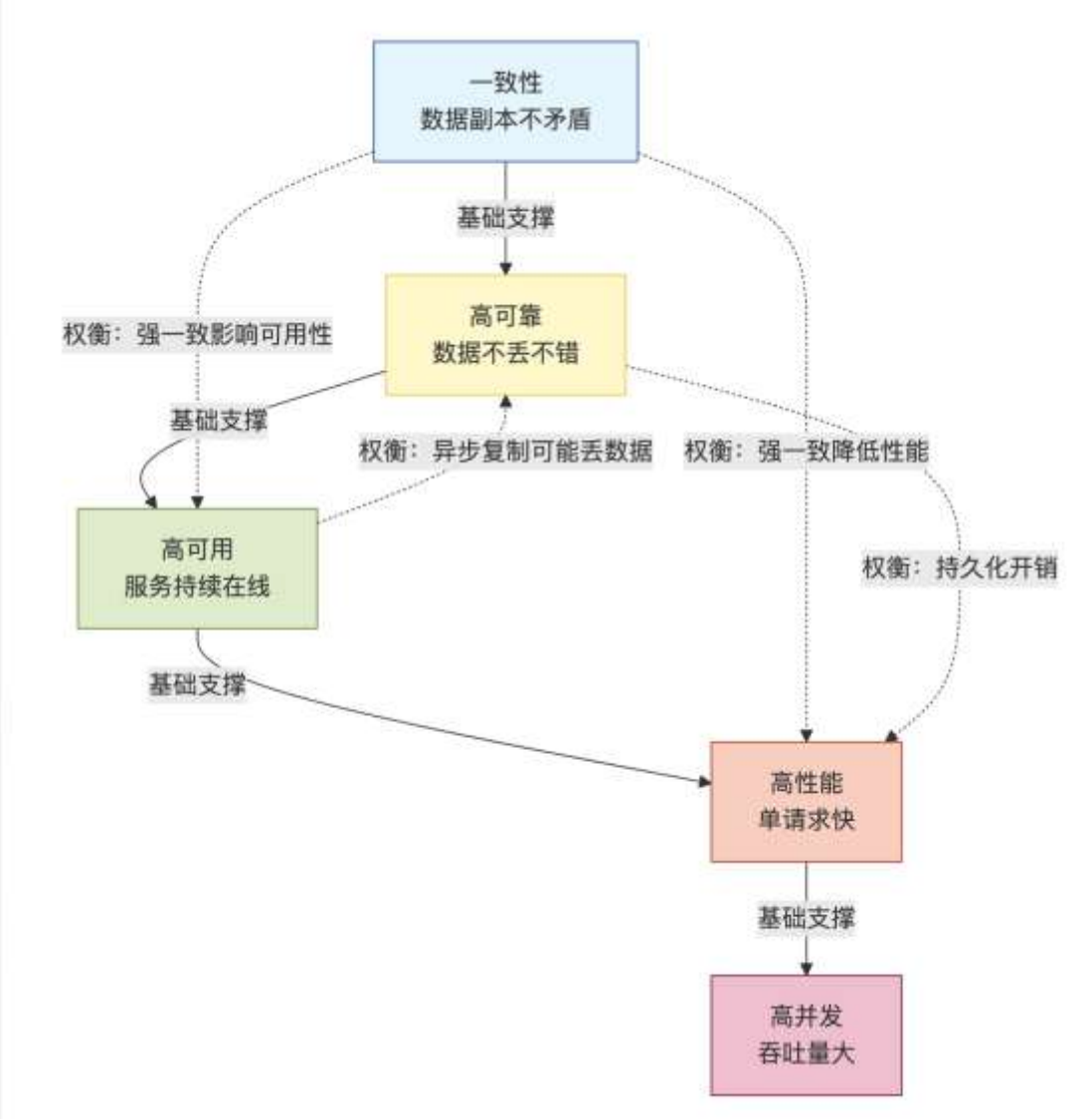
常见问题

- **常见问题1：**
  - Q： “CAP和BASE有什么关系？”
  - A： “CAP是理论边界，BASE是AP系统的实践指南。当我们选择AP时，就用BASE原则设计系统。”
- **常见问题2：**
  - Q： “举一个你设计的最终一致性系统的例子？”
  - A： “我设计过一个评论系统，用户发表评论后立即显示（基本可用），但评论数统计异步更新（最终一致），这样保证了高并发下的用户体验。”
- **常见问题3：**
  - Q： “什么场景不能用最终一致性？”
  - A： “金融交易的核心链路（如扣款）、医疗系统的关键操作、法律文件的签署等，这些需要强一致性保证。”

4、 分布式软件的关键问题

4、 分布式软件的关键问题

- **核心目标**
  - 高并发
  - 高性能
  - 高可用
- **基本要求**
  - 高可靠
  - 一致性



武漢大學
WUHAN UNIVERSITY

分布式软件的关键问题

概念	核心定义	关键指标	通俗理解
高并发	系统能同时处理大量请求的能力	QPS、TPS	“扛得住”
高性能	单个请求处理速度快	响应时间（RT） 延迟（p99）	“跑得快”
高可用	系统持续提供服务的能力	SLA（如99.99% = 年停机52分钟） MTBF（平均无故障时间）MTTR（平均恢复时间）	“不宕机”
高可靠	数据不丢失、计算结果正确	持久性、错误率	“不出错”
一致性	多个副本之间数据的关系	一致性模型（强、弱、最终等）	“不矛盾”

高并发

定义

- 指系统在短时间内处理大量请求的能力。
- 它关注的是“量”，即系统的吞吐量和资源调度。

衡量指标

- QPS (Queries Per Second)：每秒查询数。
- TPS (Transactions Per Second)：每秒事务数。
- 并发连接数：系统同时承载的连接总量。

场景解决方案

- 横向扩展：增加服务器集群（负载均衡）。
- 缓存策略：使用 Redis 减轻数据库压力。
- 异步处理：引入消息队列（MQ）削峰填谷。

高性能

- **定义**
  - 指系统的响应速度和资源利用效率。高性能往往是为了提升用户体验，减少“转圈圈”的时间。
- **衡量指标**
  - RT (Response Time)：响应时间（毫秒级）
  - Latency：延迟
  - 资源利用率：CPU、内存、I/O 的占用情况
- **解决方案**
  - 算法优化：选择更高效的数据结构
  - 代码优化：异步IO、并发编程、锁粒度拆分
  - 硬件加持：使用 SSD、增加带宽

高可用

定义

- 指系统在面对组件故障（如服务器宕机）时，依然能提供服务的能力。核心在于“容错”和“无单点”。

衡量指标

- SLA (Service Level Agreement)：如“5个9” （99.999%），意味着全年停机时间不超过 5 分钟。
- MTBF / MTTR：平均无故障时间和平均修复时间。

解决方案

- 多机房容灾：异地多活架构。
- 故障自动切换：Keepalived + VIP 或 Zookeeper 选主。
- 服务降级/熔断：在极端情况下关闭非核心业务，保住主流程。

高可靠

定义

- 系统正确完成任务的能力。要求数据不丢失、计算结果正确
- 高可靠以高可用为基础，其内涵包括了一致性。

核心场景

- 数据可靠性：存进去的数据不能丢。
- 执行可靠性：指令发出后，必须有确定的结果。

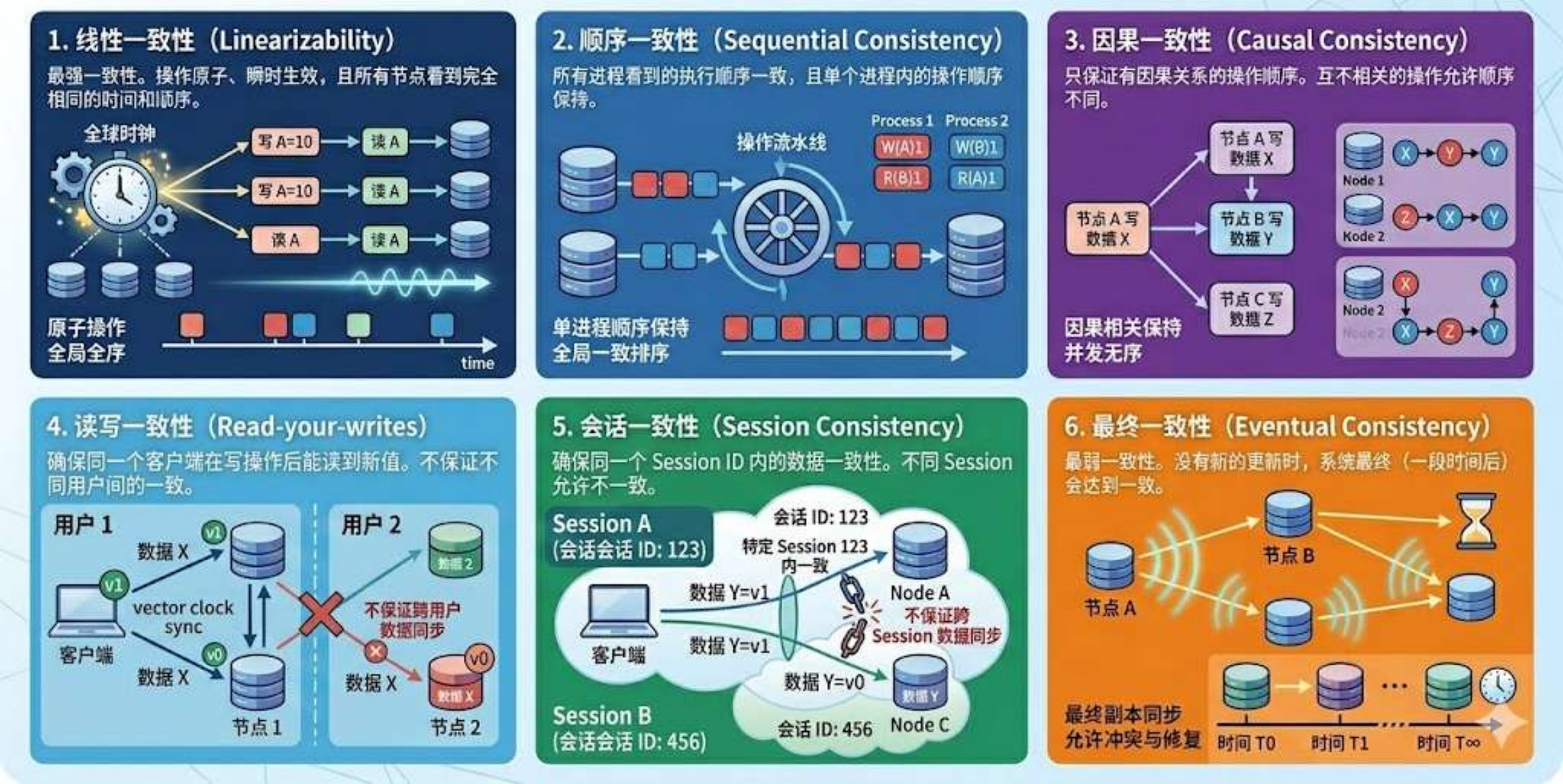
常见解决方案

- 故障隔离：故障范围最小化，防止雪崩效应；舱壁隔离、熔断、限流
- 故障转移：故障发生时无需人工介入，系统自愈　健康检查 + 选主/切换
- 容错与共识：部分节点故障不影响整体一致性，Raft、Paxos、2F+1模型
- 可观测性：实时感知系统健康状态。监控、告警、链路追踪

一致性

- **定义**
  - 多个副本之间，在同一时刻的数据是否达成共识。
- **三种级别**
  - 强一致性：更新后，后续访问立刻能看到最新值。
  - 弱一致性：更新后，不保证后续访问能看到最新值。
  - 最终一致性：不保证实时，但经过一段时间后，所有副本数据会趋于一致。
- **常见方案**
  - 强一致性方案：2PC、3PC、Paxos、Raft、ZAB
  - 最终一致性方案：读写Quorum、 Gossip协议、矢量时钟、版本向量
  - 分布式事务：XA、TCC、Saga、本地消息表、事务消息
  - 其他：分布式锁（Redis/ZooKeeper）、MVCC、一致性哈希

一致性模型


1. 线性一致性（Linearizability）
最强一致性。操作原子、瞬时生效，且所有节点看到完全相同的时间和顺序。
全球时钟
写 A=10
读 A
写 A=10
读 A
读 A
读 A
原子操作
全局全序
time
2. 顺序一致性（Sequential Consistency）
所有进程看到的执行顺序一致，且单个进程内的操作顺序保持。
操作流水线
Process 1
Process 2
W(A)1
W(B)1
R(B)1
R(A)1
单进程顺序保持
全局一致排序
3. 因果一致性（Causal Consistency）
只保证有因果关系的操作顺序。互不相关的操作允许顺序不同。
节点 A 写数据 X
节点 A 写数据 X
节点 B 写数据 Y
节点 C 写数据 Z
Node 1
Node 2
Node 2
因果相关保持
并发无序
4. 读写一致性（Read-your-writes）
确保同一个客户端在写操作后能读到新值。不保证不同用户间的一致。
用户 1
用户 2
数据 X
v1
v1
vector clock sync
客户端
v0
数据 X
节点 1
数据 2
不保证跨用户数据同步
数据 X
v0
数据 X
节点 2
5. 会话一致性（Session Consistency）
确保同一个 Session ID 内的数据一致性。不同 Session 允许不一致。
Session A
(会话会话 ID: 123)
会话 ID: 123
特定 Session 123 内一致
数据 Y=v1
Node A
不保证跨 Session 数据同步
客户端
数据 Y=v1
数据 Y=v0
Session B
(会话会话 ID: 456)
会话 ID: 456
Node C
6. 最终一致性（Eventual Consistency）
最弱一致性。没有新的更新时，系统最终（一段时间后）会达到一致。
节点 A
节点 B
最终副本同步
允许冲突与修复
时间 T0
时间 T1
时间 T∞

强一致性

- **线性一致性 (Linearizability)**
  - 操作一旦完成，其效果在真实时间的这一刻起，所有后续读操作都能看到最新值。
  - 例：分布式锁服务（etcd、ZooKeeper）、支付宝余额更新
- **顺序一致性 (Sequential Consistency)**
  - 所有进程看到的操作顺序完全一致，但这个顺序与真实时间无关，只要每个进程自身操作顺序被保留
  - 例：消息队列、HDFS 的文件元数据操作

维度	线性一致性	顺序一致性
全局统一顺序	✅ 是	✅ 是
实时性	✅ 实时生效	❌ 可延迟
CAP归类	CP系统	CP系统
强弱归属	强一致	强一致

弱一致性

- **因果一致性 (Causal Consistency)**
  - 有因果关系的操作必须按因果顺序被所有节点看到，而无因果关系的并发操作允许以任意顺序出现。
  - 例：微信聊天、社交网络评论/回复系统、腾讯文档协同编辑
- **读写一致性 (Read-Your-Writes Consistency)**
  - 用户总是能读到自己刚写入的最新值，但其他用户可能暂时看不到。
  - 例：修改头像昵称、购物车、评论/点赞后立即显示
- **会话一致性 (Session Consistency)**
  - 在同一个会话连接中，读写操作满足读写一致性；不同会话间不做保证。
  - 例：Web购物下单、在线游戏的状态

弱一致性

- **单调读一致性 (Monotonic Read Consistency)**
  - 用户不会读到数据时光倒流——即如果读到了某个值，后续读操作绝不会读到比它更旧的版本。
  - 例：用户查看朋友圈、微博时间线的新内容应出现在旧内容之后
- **单调写一致性 (Monotonic Write Consistency)**
  - 同一用户的写操作会按提交顺序被所有副本应用，后发的写不会先于前一个写生效。
  - 例：积分增减、等级提升
- **最终一致性 (Eventual Consistency)**
  - 如果系统不再接收新的更新，经过一段时间（不保证时长），所有副本最终会收敛到相同的值
  - 例：DNS解析、普通商品库存（非秒杀）更新、微博的粉丝数、视频播放量

属性之间的依赖关系

上层属性	依赖的下层属性	原因
高并发	高性能、高可用	单请求慢则无法支撑高并发；服务不可用则并发为0
高性能	高可靠、一致性	数据不一致导致重试或错误处理，拖慢性能
高可用	高可靠、一致性	故障恢复时需要从一致的数据副本重建服务
高可靠	一致性	副本不一致则数据可靠无从谈起

作业：商品库存与秒杀系统设计

系统设计文档

- 绘制系统架构草图（服务拆分：用户服务、商品服务、订单服务、库存服务）
- 定义各服务API接口（RESTful）
- 数据库ER图（用户表、商品表、库存表、订单表）
- 技术栈选型说明（编程语言、框架、中间件初选）

环境准备

- 初始化项目代码仓库（Git）
- 搭建基础开发环境（如Spring Boot + MyBatis + MySQL）
- 搭建一个项目代码框架，实现简单的用户注册登陆功能